Auditr Viva Questions and Answers

A detailed Q&A; bank based on the current codebase, architecture, and deployed model metadata.

Tip: Do not memorize word for word. Understand the structure and answer naturally.

Question bank

Q1. What is Auditr?

Suggested answer: Auditr is a Streamlit-based audit intelligence workspace that accepts ledger CSV files, validates them, engineers transaction-risk features, scores suspicious transactions using XGBoost, adds anomaly-assisted prioritization, and explains flagged cases in an auditor-friendly way.

Q2. Why did you choose this topic?

Suggested answer: We chose it because fraud detection and audit intelligence are important in finance, but many demos focus only on the machine-learning score. We wanted to solve the full practical workflow, including messy CSV intake, prioritization, explainability, and review support.

Q3. What makes your project different from a basic fraud model?

Suggested answer: The project is not just a classifier. It includes project-based workflow, CSV validation, quarantine of bad rows, anomaly-assisted prioritization, explainability, reviewer feedback, and auditor-facing pages.

Q4. What is the main model used in the app?

Suggested answer: The main deployed classifier is XGBoost. Its readable configuration is in backend/models/xgb_model.py, while the trained runtime artifact is stored as backend/models/artifacts/xgb_model_bundle.pkl.

Q5. Why did you choose XGBoost?

Suggested answer: XGBoost performs strongly on structured tabular data, works well with feature importance and contribution-based explainability, and benchmarked strongly against the other candidate models in this project.

Q6. What other models did you compare against?

Suggested answer: We benchmarked XGBoost against HistGradientBoosting, RandomForest, and ExtraTrees before selecting XGBoost as the deployment model.

Q7. How was the model evaluated?

Suggested answer: It was evaluated using a chronological train-validation-test split of 70%, 15%, and 15%. The current saved metadata shows a test ROC-AUC of 0.989 and a test F1 score of 0.937.

Q8. Why use chronological holdout instead of random split?

Suggested answer: Because transaction history is time-sensitive. A random split can leak future behavior into past training, which gives unrealistically optimistic results. Chronological holdout is more realistic for audit use cases.

Q9. What does the .pkl file contain?

Suggested answer: It contains the trained XGBoost model object and metadata used by the live app. It is only the saved runtime artifact, not the human-written source code.

Q10. If the professor asks to see the model code, what will you show?

Suggested answer: We will show backend/models/xgb_model.py for the readable model definition and backend/training/train_auditr_model.py for the training and evaluation logic.

Q11. What is the role of backend/utils.py?

Suggested answer: It is the main backend engine. It handles TOTP authentication, session state, project storage, CSV parsing, quarantine, feature engineering, anomaly scoring, XGBoost inference, explainability, and chart helpers.

Q12. How do you handle messy CSV files?

Suggested answer: The system tries multiple encodings and separators, normalizes headers, maps aliases, validates required fields, and quarantines rows with invalid dates, invalid amounts, or missing vendors. Only clean rows continue to scoring.

Q13. What happens to quarantined rows?

Suggested answer: They are stored separately so the auditor can download and correct them later. This prevents broken data from contaminating the model input.

Q14. What features did you engineer?

Suggested answer: We engineered amount-based, invoice-based, timing-based, control-based, and novelty-based features such as amount deviation, vendor average amount, duplicate invoice signals, same-day bursts, approval-threshold crossing, and new vendor behavior patterns.

Q15. Why did you add anomaly detection?

Suggested answer: Because some suspicious transactions may look unusual even if they do not match previously observed fraud patterns exactly. The anomaly score adds extra prioritization context.

Q16. Which anomaly method did you use?

Suggested answer: The code uses Isolation Forest when available and enough rows exist. If not, it falls back to normalized z-score logic across the numeric feature set.

Q17. How do you combine model score and anomaly score?

Suggested answer: We compute a blended_risk_score using fraud probability, anomaly score, and a small control-intensity signal. That becomes the blended_priority_score used to rank the manual review queue.

Q18. How is explainability achieved?

Suggested answer: We use XGBoost contribution values from the booster to estimate per-feature impact on the score. The backend then converts the strongest driver into a human-readable summary and suggested next step for the auditor.

Q19. Is a flagged transaction equal to fraud?

Suggested answer: No. A flagged transaction is only a prioritization signal. It means the row looks unusual enough to require manual review, not that fraud is proven.

Q20. What does the Projects page do?

Suggested answer: Projects is the main workflow page. It creates engagements, uploads CSVs, previews schema mapping, shows data-quality metrics, opens saved projects, and can load the demo pack.

Q21. What does the Dashboard page do?

Suggested answer: It gives the auditor an overview of the active engagement, including review counts, flagged reasons, department concentration, vendor exposure, control signals, the manual review queue, and extra risk intelligence.

Q22. What does the Transactions page do?

Suggested answer: It acts as a ledger explorer with filtering, searching, thresholding, reviewer feedback updates, and export options for cleaned reports or flagged rows.

Q23. What does the Explainability page do?

Suggested answer: It focuses on one flagged transaction at a time and shows its risk score, main reason, supporting signals, case facts, vendor evidence, and reviewer decision flow.

Q24. How do you store projects?

Suggested answer: Project metadata and case feedback are stored in SQLite under auditr_workspace/auditr.db, while uploaded ledgers and related per-project files stay inside auditr_workspace/projects.

Q25. What are the main limitations?

Suggested answer: The data is synthetic, the app is a local demo rather than a multi-user production system, and the model is not retrained live inside the app.

Q26. What future work would improve this project?

Suggested answer: Evidence attachments, cross-project intelligence, supervisor-ready report packs, stronger multi-user review workflow, and cloud deployment would all strengthen the project.

Q27. How was the work divided between the two of you?

Suggested answer: Abdul focused mainly on backend and model logic such as feature engineering, scoring, evaluation, and persistence. Maryam focused mainly on frontend workflow, user-facing audit flow, testing, and presentation/documentation. Final integration and validation were shared.